

从 Copilot 到 Agentic Loop

AI 工具落地与基础设施搭建经历

From Copilot to the Agentic Loop
Delivering AI Tools and Building the Supporting Infrastructure

马铭泽 · 馬銘澤 · Mingze Ma

金融风险、系统与数据控制 · Agent 工具落地 · 能力与知识底座

ECL 项目交付	审计回应架构	查询工具与 Wiki	Bono Insight
----------	--------	------------	--------------

LANGUAGE / 语言 / 語言

[简体中文](#) [繁體中文](#) [English](#)

项目网站 · 項目網站 · PROJECT WEBSITE

<https://bono12138.github.io/mingze-ma-agentic-loop-portfolio/>

点击打开三语言网站 · 掃碼查看 · Click to open



扫码查看 · SCAN

Prepared 6 August 2026 · 用于求职沟通与项目经验说明

目录 · 目錄 · Contents

简体中文 · 马铭泽

我的工作背景

- 一、从“帮我写代码”到参与一轮 ECL 更新
- 二、审计回应架构：把问题处理设计成一条可以反复运行的管道
- 三、从 ECL 项目中抽出可复用的 Superset 查询工具
- 四、从内部使用进入更高一层的架构设计
- 五、“大基建”：Bono Agent 能力与知识底座
- 六、Bono Insight：把方法做成可安装插件
- 七、分享与其他快速落地案例
- 八、流程从哪里来

繁體中文 · 馬銘澤

工作背景

- 一、從「幫我寫程式碼」到參與一輪 ECL 更新
- 二、審計回應架構：把問題處理設計成一條可以反覆運行的管道
- 三、從 ECL 項目中抽出可重用的 Superset 查詢工具
- 四、從內部使用進入更高一層的架構設計
- 五、「大基建」：Bono Agent 能力與知識底座
- 六、Bono Insight：把方法做成可安裝插件
- 七、分享與其他快速落地案例
- 八、流程從哪裏來

English · Mingze Ma

Professional background

- 1. From generating code to supporting an ECL update
- 2. Audit-response architecture: a repeatable issue-resolution pipeline
- 3. Extracting a reusable Superset query tool from the ECL work
- 4. From internal use to a higher level of architecture
- 5. The Bono Agent capability and knowledge foundation
- 6. Bono Insight: packaging the methods as a plugin
- 7. Knowledge sharing and other rapid delivery cases
- 8. Where processes come from

从 Copilot 到 Agentic Loop

AI 工具落地与基础设施搭建经历

我的工作背景

我的工作一直围绕金融风险、系统、数据和流程展开。我曾在安永金融服务科技风险团队工作，参与银行、保险和证券项目的 ITGC、ITAC 以及系统与数据控制审阅；目前在 Akulaku Group 的工作包括 ECL 和操作风险管理。ECL 工作涉及模型方法、数据处理、系统需求、UAT、财务对账、模型更新和外部审阅支持；操作风险管理则需要持续梳理业务流程及其中的控制安排。

我最初使用 AI，只是为了完成一些具体工作：生成代码片段、编写查询脚本。后来，它开始参与模型、数据、审计回应和工具开发。随着任务链变长，我关注的也不再只是一次输出，而是业务背景如何保留、工具如何衔接，以及这次遇到的问题能不能改变下一次执行。

这几年，我使用 AI 的方式已经变了几次。2023 年，我主要使用页面端 AI 工具生成代码片段，辅助开发自动化工具和查询脚本。随后，我开始使用 Cursor、Antigravity 和 Trae 等 IDE，让 AI 以 Copilot 的方式参与开发。从 2025 年开始，Claude Code、Cursor CLI、Codex 和 Hermes 等 Agent 逐渐参与更长、更复杂的任务，包括模型与工具开发、自动化流程构建、调研和文档编写。

到 2026 年，为了形成持续运行的 Agentic Loop，我开始把工作重心转向 Agentic Engineering。基于此前积累的业务知识和数据资料，以及内控和流程梳理经验，我对已有工具和自动化流程进行解耦与重新架构，调整交互流程和展示方式，并根据公司安全要求和实际业务需要分阶段开放相关能力。本文以这段时间的几个实际项目为节点，记录我从 Copilot 走向 Agentic Engineering，以及在现有组织环境中落地 Agent 自动化的过程。



图 1 从 Copilot 到 Agentic Loop 的工作方式变化

一、从“帮我写代码”到参与一轮 ECL 更新

早期使用 ChatGPT 时，我通常给出一个具体需求，例如合并 Excel、处理 Word 表格、采集公开信息或编写一段数据处理脚本。AI 负责生成代码，我负责复制、运行、报错后再修改。它能减少写代码的时间，却看不到完整项目，也无法自己检查上下游文件是否一致。

Cursor、Claude Code 和 Codex 进入代码仓库以后，任务的粒度开始变大。2026 年 6 月 30 日集团 ECL 更新是我目前最完整的一次实践。项目覆盖印度尼西亚和菲律宾的四个贷款组合，包括印尼 BNPL、印尼 Cash、菲律宾 BNPL 和印尼生产性贷款。工作从财务范围和 EAD 业财核对开始，继续完成订单级数据构造、PD、LGD、EAD、Stage、前瞻性调整、ECL 汇总、结果分析、方法说明和历史审计问题答复。

这一轮工作没有一个现成平台可以从头跑到尾。数据来自 Superset/Hive 和财务文件；FY25 已审阅模型是方法基线；部分加工需要在服务器端完成；Python 和 Notebook 负责计算与展示；Excel 承担结果阅读、复核和汇报；Git 保存代码、规则和版本。Agent 需要在这些位置之间持续工作，生成 SQL 只是其中一个步骤。

我负责确认报告日人口、方法变化、数据口径、重大操作和最终验收。Agent 承担了大量连续执行工作：查找表和字段、编写并运行 SQL、处理资源错误、构造中间数据、准备模型输入、运行模型、比较历史结果、生成工作簿和整理审阅材料。涉及既有表结构修改、删除或结果不确定的写入，仍由我确认准确对象后再继续。

为了让项目可以复查，我要求每次正式查询保留 SQL、查询编号、运行时间、行数、结果文件和文件哈希；模型输入要冻结；写入后立即执行控制检查；请求超时后先检查目标状态，不能因为客户端没有收到结果就盲目重跑。四个产品的正式 Notebook、集团结果、方法文档和审计问题答复，也必须能从同一套输入重新生成。

这次项目让我第一次把 Agent 当作持续执行者来管理。任务粒度从一个函数扩大到一段完整工作：沿着已经确认的范围把取数、计算、检查和交付继续做下去。

相关产物：项目执行总控、数据构造 SQL、分组合 Notebook、集团结果、结果分析工作簿、方法文档、历史审计问题答复和自动验收脚本。

二、审计回应架构：把问题处理设计成一条可以反复运行的管道

这套架构在 6 月 30 日项目之前已经形成。2026 年 3 月至 5 月，我在 FY25 ECL 审计回应中发现，一条审计问题往往会同时影响方法、代码、数据、说明文档和最终交付。如果 Agent 收到问题后立即修改，调查、判断、实施和复核很容易混在一起，解决一个问题的同时又制造新的不一致。

我重新设计了整套审计回应架构，把工作分成四个相互衔接、但不能互相替代的阶段：

每个阶段都有自己的输入、允许执行的操作、需要留下的证据和退出条件。调查阶段不能顺手改代码，Agent 也不能用“已经修改”代替对问题的判断；涉及方法选择和重要影响时，由我作出决定。阶段边界被写进仓库规则、脚本、测试和状态文件，不只依赖一次对话里的提示。

这套架构也是“管道理论”在实际项目中的体现。Agent 是执行节点，任务背景、数据、方法、权限、验证和反馈组成周围的管道。实际运行中出现的问题会继续加固这条管道：缺少修改前基线，就补上强制对比；不同模型之间发生状态污染，就改用隔离运行；Notebook 看起来在计算但结果没有真正使用计算值，就加入依赖链检查和扰动测试。

后来出现的 AQ 编号、处理窗口和排队规则，都是这套架构持续运行后，为解决状态保存、并行问题和中途打断而增加的功能，并不是设计的起点。6 月 30 日项目中的任务状态、基线、重跑、对账和验收，也直接沿用了这套架构。

相关产物：审计回应总体架构、配套 SOP、执行规则与检查脚本、基线及影响对比、伪计算检测、独立终审和可视化流程图。



图 2 审计回应架构：调查、决策、实施与终审

三、从 ECL 项目中抽出可复用的 Superset 查询工具

ECL 更新和资产质量分析都需要反复查找数据、编写 SQL、运行查询、处理报错和保存结果。Agent 当时已经可以协助理解问题和编写 SQL，但查询仍要由人工手工复制到 Superset，执行过程和返回结果也无法自然接回后续工作。同一套能力因此散落在不同项目里，难以复用。

我把查询执行从具体的 ECL 项目中抽出来，设计并开发了 query-superset。Agent 负责理解业务问题和准备 SQL，工具负责通过公司原有的 Superset 身份和权限执行只读查询，再把错误信息、查询结果和运行记录返回给 Agent。整个方案继续使用公司已有的数据访问通道，没有为自动化另外开放数据库直连。

我负责需求梳理、架构设计、安全边界、脚本开发、测试和安装方式，并把它整理成可以在不同 Agent 项目中复用的独立 Skill。真实环境测试覆盖了登录、查询执行、常见错误处理和结果保存，工具随后被打包并配套内部使用说明。

这一步让我第一次从一个完整业务项目中抽出独立的原子能力。Agent 不再只负责“写一段 SQL”，而是可以继续完成实际查询并接收结果。工具随后通过内部 Wiki 和安装包向公司同事开放。

相关产物：query-superset Skill、Windows 查询工具、测试、安装包、使用说明和能力边界文档。

四、从内部使用进入更高一层的架构设计

工具发布以后，多个不同部门的同事开始在实际工作中使用。内部 Wiki 和配套材料除了提供安装与使用方法，也说明了工具的适用范围、公司现有权限与安全控制，以及使用过程中需要注意的问题。

随着使用范围扩大，大家提出的问题很快从“怎样查到数据”延伸到数据清洗、数据分析、业务知识和结果交付。查询工具解决了数据取得，却不能独立决定分析口径、选择调查方法或解释结果。继续增加查询功能已经无法回答这些问题。

我开始重新梳理数据查询、数据分析与业务判断之间的关系，并把执行工具、分析方法和事实依据分开。这一步把项目带到了更抽象的架构层，也形成了后来“大基建”的设计方向。

WINDOWS	SUPerset	CORSOR SKILL	READ ONLY
复用现有权限 继续走 Superset、公司网络和员工自己的账号，不另开数据库直连。		保持只读边界 只执行一条只读 SQL，保留结果、错误和查询编号。	把安装交给 Cursor 员工只需下载、解压并复制部署指令，Cursor 完成配置和实测。

证据摘录 Superset 查询工具的权限与只读边界

CONFIDENCE	HOW	ACCESS CONTROL	TEST EVIDENCE
① 这份长文记录方案背后的治理判断：为什么选择复用现有通道、哪些控制可以复用、哪些不能想当然，以及测试证据目前能证明什么。			

证据摘录 从内控与治理经验出发说明设计依据

为什么 Wiki 不应该只靠手工复制粘贴

手工维护 Wiki 的问题：

- 格式不一致：每次粘贴排版都不同
- 版本失控：不知道哪个版本是最新的
- 无法审计：谁改了什么，为什么改
- 无法批量更新：几十个页面的统一修改要手动逐页操作
- 敏感信息泄露风险：手工复制容易带入内部数据

证据摘录 内部 Wiki 的工程化发布与更新

“试跑了一条，通了。”

内部同事完成真实查询后的反馈。后续讨论很快从取数延伸到数据分析、MCP 和业务信息输入。

真实使用反馈及随后出现的分析需求

五、“大基建”：Bono Agent 能力与知识底座

2026 年 7 月，我把前面的实践重新整理成一个内部项目，当时简称“大基建”，正式名称是“Bono Agent 能力与知识底座”。它不再围绕某一个工具扩充功能，而是重新划分 Agent 完成业务任务时需要的不同组成部分。

整个设计最后形成四层：

任务层：业务任务仍由项目本身负责

任务层目前承载集团 ECL、PD Assessment 和月度预测等具体工作。每个项目继续保留自己的任务入口、AGENTS.md、执行总控和交接状态，不把不断变化的任务信息复制进通用插件。insight-router 作为统一入口，先锁定原始问题、对象、期间、人口、读者、输出和风险，再选择完成任务所需的最小路线。

每次调用会生成一个 usage_id，把本次任务实际使用的 Skill、结果和反馈串在一起。任务结束时，Router 分别处理项目状态、方法或工具问题以及知识候选，避免“更新了一个 Skill”或“记录了一条反馈”被误当成业务任务已经完成。下一阶段需要验证的是，新用户能否只依靠项目入口和已有说明完成任务，而不需要我在旁边持续解释。

方法层：把反复使用的工作方法做成 Skill

方法层已经形成一组相互衔接的能力：prepare-analysis-data 负责人口、期间、粒度、主键、映射、控制数和数据快照；investigate-data-change 负责异常和趋势调查、正负分支、竞争性解释和停止条件；deliver-analysis 负责把已验证结果写成业务人员可以使用的说明；deliver-workbook 负责 Excel 的阅读顺序、数字语义、证据入口和交付检查；insight-router 负责跨层路由和任务收尾。

这些 Skill 不只是提示词。每项能力都配有参考方法、任务模板和完成标准；变化调查另有独立的深度验证脚本，用来检查原问题是否被保留、增加和减少是否都已覆盖、原因能否加回总变化，以及停止调查是否有足够证据。当前路线图优先在 ECL、PD Assessment、资产质量调查和 Excel 交付中验证现有方法，后续再根据重复出现的业务需求和整体架构补充新的方法 Skill。

原子能力层：复用已有工具，不在插件里重复开发

原子能力层负责执行边界明确的动作。现有能力包括 Superset 查询、公司 Wiki 读取、本地文件和浏览器操作，以及 Codex 原生的文档和 Excel 能力。Bono Insight 重点管理调用时机、证据留存和完成判断，具体执行则复用宿主环境中已有的数据库、浏览器和 Excel 能力。

例如，query-superset 继续使用公司原有的身份、权限和查询通道；deliver-workbook 负责工作簿怎样组织和验收，真正的文件创建、编辑和重算仍交给原生 Excel 能力。某项实时服务是否需要进一步封装为 MCP，会结合跨项目复用需求、权限安排、维护成本和故障恢复要求评估。

依据层：把业务事实与使用方法分开维护

依据层已经积累了业务知识入口、数据字典、源文件、查询结果、运行记录、读者偏好和项目证据。公司业务事实没有被打包进公开插件，而是继续保存在获准的宿主项目中，通过 `business-knowledge` 等能力按需读取。每条重要知识除了说明“是什么”和“在哪里”，还需要记录适用主体、粒度、有效期间、来源、当前状态和确认程度，避免 Agent 把旧口径当成当前事实。

当前阶段使用 Git 管理结构化文本和原始证据，可以满足现有规模下的版本、来源和有效期间管理。一次任务发现的新事实先进入“知识候选”，只有权威条目真正更新并完成核实后，才算知识已经应用。随着团队使用范围和实时调用需求扩大，可以继续评估向量数据库或长期知识服务；近期计划则包括建立更明确的业务上下文接口，并为正式证据可重建缓存和临时结果设置不同的留存与清理规则。

四层怎样连接起来

四层通常从任务向下调用，证据和状态再向上返回：Router 先锁定任务合同，选择所需方法；方法调用宿主项目批准数据库、Wiki、文件或 Excel 能力；执行结果与来源进入依据层；最后回到原始问题完成验证和交付。Workbench 是实际工作的承载平台，AGENTS.md 负责跨层安全和治理规则，Plugin 负责安装和发布，它们不再被包装成额外的业务层。

系统还配有本机反馈记录器，将能力问题和知识候选分开进入各自的状态流程；公开 GitHub Issue 只能由脱敏且经过人工复核的内容生成。仓库中的自动测试则检查插件结构、反馈状态、公开内容脱敏和调查深度。这样，一次任务中的纠正才有机会进入下一次使用，同时又不会让 Agent 自动改写规则并直接发布。



图3 Bono Agent 能力与知识底座的四层设计

回到 0630：这套架构怎样进入实际交付

这套设计并不是在业务项目之外搭好以后才投入使用。服务于集团 IPO 相关财务准备的 0630 ECL 更新，是它目前最完整的一次实际运行。前面已经积累的 Superset 查询、业务知识、数据准备方法、审计回应管道和验收规则，让 Agent 可以沿着同一个任务入口，连续推进四个贷款组合的数据构造、模型运行、交叉核对、异常调查、结果工作簿和方法材料。已经确认过的字段含义、方法基线、文件位置和完成标准不需要在每轮对话中重新说明，查询、模型输入、运行结果和复核证据也能沿同一条链路保存。这些基础能力正在支撑本轮模型与数据完整更新的快速交付。

0630 也暴露了原有设计中仍然粗糙的地方，并直接推动了任务层和方法层继续成形。四个组合共同更新，要求任务层更严格地固定报告日、人口、组合范围、方法版本、重大操作和验收状态；人口核对、日期语义、主键、映射和数据快照进入了 `prepare-analysis-data`；产品和 Bucket 变化不能只做净额解释，推动 `investigate-data-change` 增加正负分支、量化归因、竞争性解释和停止条件；Notebook、集团结果和工作簿需要由未参与开发的人复核，则继续补充了 `deliver-analysis` 和 `deliver-workbook` 的交付标准。

同一轮项目还在补充依据层。四个组合的业务定义、数据来源、字段限制、已审阅方法、财务控制数、正式查询和运行记录，被放回各自的权威位置，并标明适用期间和确认状态。这样形成的是一个来回工作的循环：已有架构减少项目中的重复查找和返工，项目里真正遇到的问题再进入相应的任务入口、方法 Skill 或知识依据，成为下一轮执行的起点。



图 4 0630 项目对四层架构的反馈回路

六、Bono Insight：把方法做成可安装插件

经过 0630 等实际任务验证后，我把已经稳定下来的通用方法、模板、验证脚本和安全边界整理成 Bono Insight，没有把整个 Workbench、公司业务知识或私有数据能力一起塞进插件。当前版本以 OpenAI/Codex Preview 插件发布，其他平台如需支持，计划采用薄适配层，共享同一套核心方法，而不是复制多份 Skill 正文。

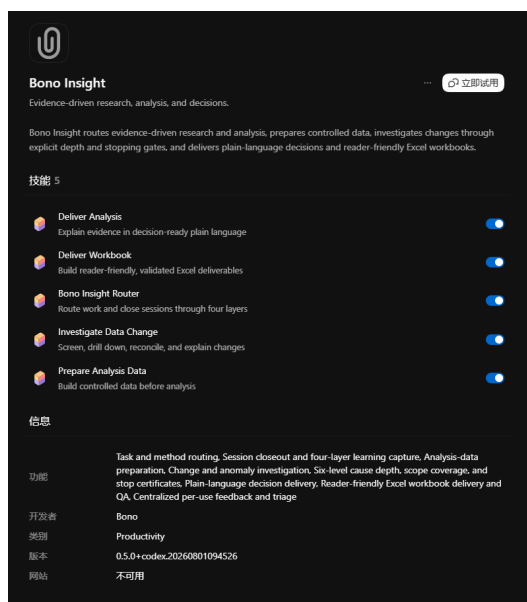
Bono Insight 已经整理出可安装的 Skill、参考方法、验证脚本、安全说明和反馈机制。发布包由构建脚本从内部权威来源生成，并通过 Source Map 记录公开内容与内部方法的对应关系，避免两套版本各自演变。插件通过本地测试和 GitHub Actions 后发布，目前已经可以在 Codex 中安装和调用。

插件化以后，四层架构也有了一条持续迭代的反馈链。Router 会为每次使用建立独立记录；任务结束时，实际调用过哪些 Skill、任务是否完成、哪里出现能力问题、是否发现新的知识候选，都会进入本机反馈队列。配套定时任务按周期汇总这些记录，把重复问题、未关闭任务、待核实知识和需要复核的架构问题整理出来。

反馈进入队列后不会直接改写插件。任务范围或验收方式有问题，回到任务层修改项目入口；反复出现的数据准备、调查或交付问题，进入方法层，修改相应 Skill、参考方法或验证器；查询、Wiki、文件和 Excel 操作中的故障，回到对应的原子能力处理；新发现的业务事实则先作为知识候选，核实来源、期间和适用范围后才进入依据层。正式修改还要经过测试、版本记录和重新发布。这里所说的“自我优化”有明确边界：系统负责自动留痕、定期归集和提示待办，人负责判断哪些问题应该进入正式版本。

当前版本仍标记为 Preview。稳定版之前，重点是新电脑安装、非作者用户冷启动、跨任务连续复用、Router 是否选择了最短完整路线，以及升级、回滚、反馈留存和清理。还需要继续验证：这套定期整理和人工复核机制能否真正减少重复错误，而不是只积累更多维护事项。后续候选方向包括其他 Agent 平台的薄适配、更多脱敏示例和评测，以及在重复需求得到证据后评估实时服务或 MCP。新增 Skill 也会结合重复出现的业务需求和整体架构共同判断。

做到这里，我所说的“AI 基础设施”已经很具体：把任务、方法、执行能力和依据分开，让 Agent 知道到哪里取事实、按什么方法做、什么时候停，以及最终由谁确认。



Bono Insight 在 Codex 中的安装与使用界面

七、分享与其他快速落地案例

在完成这些项目的同时，我也会把工作中的方法整理给其他人。受一位在万科从事财务工作的朋友邀请，我协助制作了一套内部分享材料，主题是 AI 对财务工作的实际影响。内容从一项真实的资产质量报告流程展开，说明数据取得、分析、报告和复核怎样衔接，也讨论伪计算、幻觉引用、信息安全和人工判断。正式分享由这位朋友完成，我负责案例梳理、资料研究、结构和演示稿制作。

我也会用 Agent 把身边的小问题迅速做成可以运行的工具或网页：

- [BonoBox](#)：把网页、新闻和图片搜索整理成可安装的 Python/MCP 工具，包含安装、测试、错误边界和公开发布；
- [Quota Capsule](#)：读取本机 Codex 配额信息，结合近期使用速度判断本周余量，并在 macOS 菜单栏提示；
- [World Cup Rank Room](#)：把内部世界杯活动的排名、队伍比较和 AI 辅助问答做成可以直接访问的网站，并使用独立数据和公开部署。

这些项目大小不同，工作方式却很接近：先把问题说清楚，确定使用边界和验收方式，再让 Agent 加速实现、调试、测试和发布。每个项目都同时保留代码、安装方式、测试结果、失败处理、版本记录和给下一位使用者的说明。



万科财务 AI 实践分享材料（已打码）



八、流程从哪里来

我之所以持续做这些工具和基础设施，是因为几段工作经历一直把我带回同一个问题：如果流程决定了我每天在做什么，那么又是什么在塑造流程？

在安永参与审计和内控工作时，我习惯从结果向前追查：数据和证据来自哪里，控制是否真正运行，发现异常以后由谁判断和整改。现在负责 ECL 和操作风险、信息系统风险管理，一边要处理模型、数据与系统交付，一边也要持续梳理流程中的职责、交接和控制。对流程的关注不是后来使用 Agent 才出现的，它原本就是我工作的一部分。

我逐渐发现，流程很少是一次完整设计出来的。制度要求、系统限制、部门分工、过去发生的问题和个人经验，都会在里面留下痕迹。有些安排仍然必要，有些只是长期没有重新检查过。Agent 如果直接沿着这些流程加速执行，也会把其中原有的问题一并放大。

Google DeepMind 联合创始人兼 CEO 德米斯·哈萨比斯（Demis Hassabis）曾这样概括 DeepMind 创立时的使命：“Solve intelligence, and then use it to solve everything else.”——先解决智能，再用智能解决其他问题。[1]

哈萨比斯讨论的是智能能力本身：先把智能做出来，再把它用于科学和现实问题。我在工作中真正面对的，是这句话的后半段——一个组织怎样才能“用智能解决问题”。Agent 已经能够执行和推理，但组织里的流程、业务知识、权限和判断节点往往没有被说清楚。流程在这里就是智能进入组织的接口，它决定 Agent 能看到什么、可以做什么、什么时候停下来，以及对结果负责。我的工作是先把这些内容整理成 Agent 可以进入的管道，再让它沿着流程执行和复现。实际运行留下的异常、失败和反馈，又会回到任务、方法、工具或知识层，继续修改这条管道。

在实际建设中，我通常会先选取边界清楚、正在发生的业务问题，验证系统设计中的关键假设；同时保留整体架构、接口和后续扩展路径，再把经过项目和用户验证的能力沉淀为可复用的工具与方法。数据权限、职责分离、运行留痕、停止与回退、人工确认也会从架构设计阶段开始考虑。具体采用集中平台、分层能力还是其他建设方式，还需要结合组织的业务目标、现有系统、风险偏好和资源安排共同判断。

[1] Demis Hassabis, “Demis Hassabis on AI, game theory, multimodality, and the nature of creativity,” Possible, 9 April 2025: <https://www.possible.fm/podcasts/demis/>

從 Copilot 到 Agentic Loop

AI 工具落地與基礎設施搭建經歷

工作背景

我的工作一直圍繞金融風險、系統、數據和流程展開。我曾在安永金融服務科技風險團隊工作，參與銀行、保險和證券項目的 ITGC、ITAC，以及系統與數據控制審閱；目前在 Akulaku Group 的工作包括 ECL 和操作風險管理。ECL 工作涉及模型方法、數據處理、系統需求、UAT、財務對賬、模型更新和外部審閱支援；操作風險管理則需要持續梳理業務流程及其中的控制安排。

我最初使用 AI，只是為了完成一些具體工作：生成程式碼片段、編寫查詢腳本。後來，它開始參與模型、數據、審計回應和工具開發。隨着任務鏈變長，我關注的也不再只是一次輸出，而是業務背景如何保留、工具如何銜接，以及這次遇到的問題能否改變下一次執行。

這幾年，我使用 AI 的方式已經變了幾次。2023 年，我主要使用頁面端 AI 工具生成程式碼片段，輔助開發自動化工具和查詢腳本。隨後，我開始使用 Cursor、Antigravity 和 Trae 等 IDE，讓 AI 以 Copilot 的方式參與開發。從 2025 年開始，Claude Code、Cursor CLI、Codex 和 Hermes 等 Agent 逐漸參與更長、更複雜的任務，包括模型與工具開發、自動化流程構建、調研和文件編寫。

到 2026 年，為了形成持續運行的 Agentic Loop，我開始把工作重心轉向 Agentic Engineering。基於此前積累的業務知識和數據資料，以及內控和流程梳理經驗，我對已有工具和自動化流程進行解耦與重新架構，調整互動流程和展示方式，並根據公司安全要求和實際業務需要分階段開放相關能力。本文以這段時間的幾個實際項目為節點，記錄我從 Copilot 走向 Agentic Engineering，以及在現有組織環境中落地 Agent 自動化的過程。



圖 1 從 Copilot 到 Agentic Loop 的工作方式變化

一、從「幫我寫程式碼」到參與一輪 ECL 更新

早期使用 ChatGPT 時，我通常給出一個具體需求，例如合併 Excel、處理 Word 表格、採集公開信息或編寫一段數據處理腳本。AI 負責生成程式碼，我負責複製、運行、報錯後再修改。它能減少寫程式碼的時間，卻看不到完整項目，也無法自行檢查上下游文件是否一致。

Cursor、Claude Code 和 Codex 進入程式碼倉庫後，任務的粒度開始變大。2026 年 6 月 30 日集團 ECL 更新是我目前最完整的一次實踐。項目覆蓋印度尼西亞和菲律賓的四個貸款組合，包括印尼 BNPL、印尼 Cash、菲律賓 BNPL 和印尼生產性貸款。工作從財務範圍和 EAD 業財核對開始，繼續完成訂單級數據構造、PD、LGD、EAD、Stage、前瞻性調整、ECL 匯總、結果分析、方法說明和歷史審計問題覆答。

這一輪工作沒有一個現成平台可以從頭跑到尾。數據來自 Superset/Hive 和財務文件；FY25 已審閱模型是方法基線；部分加工需要在伺服器端完成；Python 和 Notebook 負責計算與展示；Excel 承擔結果閱讀、覆核和匯報；Git 保存程式碼、規則和版本。Agent 需要在這些位置之間持續工作，生成 SQL 只是其中一個步驟。

我負責確認報告日人口、方法變化、數據口徑、重大操作和最終驗收。Agent 承擔了大量連續執行工作：查找表和欄位、編寫並運行 SQL、處理資源錯誤、構造中間數據、準備模型輸入、運行模型、比較歷史結果、生成工作簿和整理審閱材料。涉及既有表結構修改、刪除或結果不確定的寫入，仍由我確認準確對象後再繼續。

為了讓項目可以覆查，我要求每次正式查詢保留 SQL、查詢編號、運行時間、行數、結果文件和文件雜湊；模型輸入要凍結；寫入後立即執行控制檢查；請求超時後先檢查目標狀態，不能因為客戶端沒有收到結果就盲目重跑。四個產品的正式 Notebook、集團結果、方法文件和審計問題答覆，也必須能從同一套輸入重新生成。

這次項目讓我第一次把 Agent 當作持續執行者來管理。任務粒度從一個函數擴大到一段完整工作：沿着已經確認的範圍把取數、計算、檢查和交付繼續做下去。

相關產物：項目執行總控、數據構造 SQL、分組合 Notebook、集團結果、結果分析工作簿、方法文件、歷史審計問題覆答和自動驗收腳本。

二、審計回應架構：把問題處理設計成一條可以反覆運行的管道

這套架構在 6 月 30 日項目之前已經形成。2026 年 3 月至 5 月，我在 FY25 ECL 審計回應中發現，一條審計問題往往會同時影響方法、程式碼、數據、說明文件和最終交付。如果 Agent 收到問題後立即修改，調查、判斷、實施和覆核很容易混在一起，解決一個問題的同時又製造新的不一致。

我重新設計了整套審計回應架構，把工作分成四個互相銜接、但不能互相替代的階段：

每個階段都有自己的輸入、允許執行的操作、需要留下的證據和退出條件。調查階段不能順手改程式碼，Agent 也不能用「已經修改」代替對問題的判斷；涉及方法選擇和重要影響時，由我作出決定。階段邊界被寫進倉庫規則、腳本、測試和狀態文件，不只依賴一次對話裏的提示。

這套架構也是「管道理論」在實際項目中的體現。Agent 是執行節點，任務背景、數據、方法、權限、驗證和反饋組成周圍的管道。實際運行中出現的問題會繼續加固這條管道：缺少修改前基線，就補上強制對比；不同模型之間發生狀態污染，就改用隔離運行；Notebook 看起來在計算但結果沒有真正使用計算值，就加入依賴鏈檢查和擾動測試。

後來出現的 AQ 編號、處理窗口和排隊規則，都是這套架構持續運行後，為解決狀態保存、並行問題和中途打斷而增加的功能，並不是設計的起點。6 月 30 日項目中的任務狀態、基線、重跑、對賬和驗收，也直接沿用了這套架構。

相關產物：審計回應總體架構、配套 SOP、執行規則與檢查腳本、基線及影響對比、偽計算檢測、獨立終審和可視化流程圖。



圖 2 審計回應架構：調查、決策、實施與終審

三、從 ECL 項目中抽出可重用的 Superset 查詢工具

ECL 更新和資產質量分析都需要反覆查找數據、編寫 SQL、運行查詢、處理報錯和保存結果。Agent 當時已經可以協助理解問題和編寫 SQL，但查詢仍要由人工複製到 Superset，執行過程和返回結果也無法自然接回後續工作。同一套能力因此散落在不同項目裏，難以重用。

我把查詢執行從具體的 ECL 項目中抽出來，設計並開發了 `query-superset`。Agent 負責理解業務問題和準備 SQL，工具負責通過公司原有的 Superset 身份和權限執行只讀查詢，再把錯誤信息、查詢結果和運行記錄返回給 Agent。整個方案繼續使用公司已有的數據訪問通道，沒有為自動化另外開放數據庫直連。

我負責需求梳理、架構設計、安全邊界、腳本開發、測試和安裝方式，並把它整理成可以在不同 Agent 項目中重用的獨立 Skill。真實環境測試覆蓋了登入、查詢執行、常見錯誤處理和結果保存，工具隨後被打包並配套內部使用說明。

這一步讓我第一次從一個完整業務項目中抽出獨立的原子能力。Agent 不再只負責「寫一段 SQL」，而是可以繼續完成實際查詢並接收結果。工具隨後通過內部 Wiki 和安裝包向公司同事開放。

相關產物：`query-superset` Skill、Windows 查詢工具、測試、安裝包、使用說明和能力邊界文件。

四、從內部使用進入更高一層的架構設計

工具發布後，多個不同部門的同事開始在實際工作中使用。內部 Wiki 和配套材料除了提供安裝與使用方法，也說明了工具的適用範圍、公司現有權限與安全控制，以及使用過程中需要注意的問題。

隨著使用範圍擴大，大家提出的問題很快從「怎樣查到數據」延伸到數據清洗、數據分析、業務知識和結果交付。查詢工具解決了數據取得，卻不能獨立決定分析口徑、選擇調查方法或解釋結果。繼續增加查詢功能已經無法回答這些問題。

我開始重新梳理數據查詢、數據分析與業務判斷之間的關係，並把執行工具、分析方法和事實依據分開。這一步把項目帶到更抽象的架構層，也形成了後來「大基建」的設計方向。

WINDOWS	SUPERSET	CORSOR SKILL	READ ONLY
复用现有权限 继续走 Superset、公司网络和员工自己的账号，不另开数据库直连。		保持只读边界 只执行一条只读 SQL，保留结果、错误和查询编号。	把安装交给 Cursor 员工只需下载、解压并复制部署指令，Cursor 完成配置和实测。

證據摘錄 Superset 查詢工具的權限與只讀邊界

SECURITY	DOC	ACCESS CONTROL	TEST EVIDENCE
① 这份长文记录方案背后的治理判断：为什么选择复用现有通道、哪些控制可以复用、哪些不能想当然，以及测试证据目前能证明什么。			

證據摘錄 從內控與治理經驗出發說明設計依據

为什么 Wiki 不应该只靠手工复制粘贴

手工维护 Wiki 的问题：

- 格式不一致：每次粘贴排版都不同
- 版本失控：不知道哪个版本是最新的
- 无法审计：谁改了什么，为什么改
- 无法批量更新：几十个页面的统一修改要手动逐页操作
- 敏感信息泄露风险：手工复制容易带入内部数据

證據摘錄 內部 Wiki 的工程化發布與更新

「試跑了一條，通了。」

內部同事完成真實查詢後的反饋。後續討論很快從取數延伸到數據分析、MCP 和業務信息輸入。

真實使用反饋及其後出現的分析需求

五、「大基建」：Bono Agent 能力與知識底座

2026 年 7 月，我把前面的實踐重新整理成一個內部項目，當時簡稱「大基建」，正式名稱是「Bono Agent 能力與知識底座」。它不再圍繞某一個工具擴充功能，而是重新劃分 Agent 完成業務任務時需要的不同組成部分。

整個設計最後形成四層：

任務層：業務任務仍由項目本身負責

任務層目前承載集團 ECL、PD Assessment 和月度預測等具體工作。每個項目繼續保留自己的任務入口、AGENTS.md、執行總控和交接狀態，不把不斷變化的任務信息複製進通用插件。insight-router 作為統一入口，先鎖定原始問題、對象、期間、入口、讀者、輸出和風險，再選擇完成任務所需的最小路線。

每次調用會生成一個 usage_id，把本次任務實際使用的 Skill、結果和反饋串在一起。任務結束時，Router 分別處理項目狀態、方法或工具問題以及知識候選，避免「更新了一個 Skill」或「記錄了一條反饋」被誤當成業務任務已經完成。下一階段需要驗證的是，新用戶能否只依靠項目入口和已有說明完成任務，而不需要我在旁邊持續解釋。

方法層：把反覆使用的工作方法做成 Skill

方法層已經形成一組互相銜接的能力：prepare-analysis-data 負責人口、期間、粒度、主鍵、映射、控制數和數據快照；investigate-data-change 負責異常和趨勢調查、正負分支、競爭性解釋和停止條件；deliver-analysis 負責把已驗證結果寫成業務人員可以使用的說明；deliver-workbook 負責 Excel 的閱讀順序、數字語義、證據入口和交付檢查；insight-router 負責跨層路由和任務收尾。

這些 Skill 不只是提示詞。每項能力都配有參考方法、任務模板和完成標準；變化調查另有獨立的深度驗證腳本，用來檢查原問題是否被保留、增加和減少是否都已覆蓋、原因能否加回總變化，以及停止調查是否有足夠證據。當前路線圖優先在 ECL、PD Assessment、資產質量調查和 Excel 交付中驗證現有方法，後續再根據重複出現的業務需求和整體架構補充新的方法 Skill。

原子能力層：重用已有工具，不在插件內重複開發

原子能力層負責執行邊界明確的動作。現有能力的 Superset 查詢、公司 Wiki 讀取、本地文件和瀏覽器操作，以及 Codex 原生的文件和 Excel 能力。Bono Insight 重點管理調用時機、證據留存和完成判斷，具體執行則重用宿主環境中已有的數據庫、瀏覽器和 Excel 能力。

例如，query-superset 繼續使用公司原有的身份、權限和查詢通道；deliver-workbook 負責工作簿如何組織和驗收，真正的文件創建、編輯和重算仍交給原生 Excel 能力。某項實時服務是否需要進一步封裝為 MCP，會結合跨項目重用需求、權限安排、維護成本和故障恢復要求評估。

依據層：把業務事實與使用方法分開維護

依據層已經積累了業務知識入口、數據字典、源文件、查詢結果、運行記錄、讀者偏好和項目證據。公司業務事實沒有被打包進公開插件，而是繼續保存在獲准的宿主項目中，通過 `business-knowledge` 等能力按需讀取。每條重要知識除了說明「是甚麼」和「在哪裏」，還需要記錄適用主體、粒度、有效期間、來源、當前狀態和確認程度，避免 Agent 把舊口徑當成當前事實。

當前階段使用 Git 管理結構化文本和原始證據，可以滿足現有規模下的版本、來源和有效期間管理。一次任務發現的新事實先進入「知識候選」，只有權威條目真正更新並完成核實後，才算知識已經應用。隨着團隊使用範圍和實時調用需求擴大，可以繼續評估向量數據庫或長期知識服務；近期計劃則包括建立更明確的業務上下文接口，並為正式證據、可重建快取和臨時結果設置不同的留存與清理規則。

四層如何連接

四層通常從任務向下調用，證據和狀態再向上返回：Router 先鎖定任務合同，選擇所需方法；方法調用宿主項目批准的數據庫、Wiki、文件或 Excel 能力；執行結果與來源進入依據層；最後回到原始問題完成驗證和交付。Workbench 是實際工作的承載平台，`AGENTS.md` 負責跨層安全和治理規則，Plugin 負責安裝和發布，它們不再被包裝成額外的業務層。

系統還配有本機反饋記錄器，將能力問題和知識候選分開進入各自的狀態流程；公開 GitHub Issue 只能由脫敏且經過人工覆核的內容生成。倉庫中的自動測試則檢查插件結構、反饋狀態、公開內容脫敏和調查深度。這樣，一次任務中的糾正才有機會進入下一次使用，同時又不會讓 Agent 自動改寫規則並直接發布。



圖 3 Bono Agent 能力與知識底座的四層設計

回到 0630：這套架構如何進入實際交付

這套設計並不是在業務項目之外搭好後才投入使用。服務於集團 IPO 相關財務準備的 0630 ECL 更新，是它目前最完整的一次實際運行。前面已經積累的 Superset 查詢、業務知識、數據準備方法、審計回應管道和驗收規則，讓 Agent 可以沿着同一個任務入口，連續推進四個貸款組合的數據構造、模型運行、交叉核對、異常調查、結果工作簿和方法材料。已經確認過的欄位含義、方法基線、文件位置和完成標準不需要在每輪對話中重新說明，查詢、模型輸入、運行結果和覆核證據也能沿同一條鏈路保存。這些基礎能力正在支撐本輪模型與數據完整更新的快速交付。

0630 也暴露了原有設計中仍然粗糙的地方，並直接推動任務層和方法層繼續成形。四個組合共同更新，要求任務層更嚴格地固定報告日、人口、組合範圍、方法版本、重大操作和驗收狀態；人口核對、日期語義、主鍵、映射和數據快照進入了 prepare-analysis-data；產品和 Bucket 變化不能只做淨額解釋，推動 investigate-data-change 增加正負分支、量化歸因、競爭性解釋和停止條件；Notebook、集團結果和工作簿需要由未參與開發的人覆核，則繼續補充了 deliver-analysis 和 deliver-workbook 的交付標準。

同一輪項目還在補充依據層。四個組合的業務定義、數據來源、欄位限制、已審閱方法、財務控制數、正式查詢和運行記錄，被放回各自的權威位置，並標明適用期間和確認狀態。這樣形成的是一個來回工作的循環：已有架構減少項目中的重複查找和返工，項目裏真正遇到的問題再進入相應的任務入口、方法 Skill 或知識依據，成為下一輪執行的起點。



圖 4 0630 項目對四層架構的反饋回路

六、Bono Insight：把方法做成可安裝插件

經過 0630 等實際任務驗證後，我把已經穩定下來的通用方法、模板、驗證腳本和安全邊界整理成 Bono Insight，沒有把整個 Workbench、公司業務知識或私有數據能力一併放進插件。當前版本以 OpenAI/Codex Preview 插件發布，其他平台如需支援，計劃採用薄適配層，共享同一套核心方法，而不是複製多份 Skill 正文。

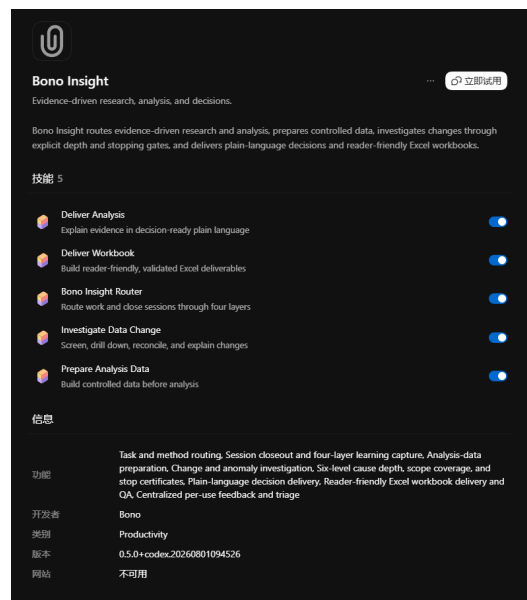
Bono Insight 已經整理出可安裝的 Skill、參考方法、驗證腳本、安全說明和反饋機制。發布包由構建腳本從內部權威來源生成，並通過 Source Map 記錄公開內容與內部方法的對應關係，避免兩套版本各自演變。插件通過本地測試和 GitHub Actions 後發布，目前已經可以在 Codex 中安裝和調用。

插件化後，四層架構也有了一條持續迭代的反饋鏈。Router 會為每次使用建立獨立記錄；任務結束時，實際調用過哪些 Skill、任務是否完成、哪裏出現能力問題、是否發現新的知識候選，都會進入本機反饋隊列。配套定時任務按周期匯總這些記錄，把重複問題、未關閉任務、待核實知識和需要覆核的架構問題整理出來。

反饋進入隊列後不會直接改寫插件。任務範圍或驗收方式有問題，回到任務層修改項目入口；反覆出現的數據準備、調查或交付問題，進入方法層，修改相應 Skill、參考方法或驗證器；查詢、Wiki、文件和 Excel 操作中的故障，回到相應的原子能力處理；新發現的業務事實則先作為知識候選，核實來源、期間和適用範圍後才進入依據層。正式修改還要經過測試、版本記錄和重新發布。這裏所說的「自我優化」有明確邊界：系統負責自動留痕、定期歸集和提示待辦，人負責判斷哪些問題應該進入正式版本。

當前版本仍標記為 Preview。穩定版之前，重點是新電腦安裝、非作者用戶冷啟動、跨任務連續重用、Router 是否選擇了最短完整路線，以及升級、回滾、反饋留存和清理。還需要繼續驗證：這套定期整理和人工覆核機制能否真正減少重複錯誤，而不是只積累更多維護事項。後續候選方向包括其他 Agent 平台的薄適配、更多脫敏示例和評測，以及在重複需求得到證據後評估實時服務或 MCP。新增 Skill 也會結合重複出現的業務需求和整體架構共同判斷。

做到這裏，我所說的「AI 基礎設施」已經很具體：把任務、方法、執行能力和依據分開，讓 Agent 知道到哪裏取事實、按甚麼方法做、甚麼時候停，以及最終由誰確認。



Bono Insight 在 Codex 中的安裝與使用介面

七、分享與其他快速落地案例

在完成這些項目的同時，我也會把工作中的方法整理給其他人。受一位在萬科從事財務工作的朋友邀請，我協助製作了一套內部分享材料，主題是 AI 對財務工作的實際影響。內容從一項真實的資產質量報告流程展開，說明數據取得、分析、報告和覆核如何銜接，也討論偽計算、幻覺引用、信息安全和人工判斷。正式分享由這位朋友完成，我負責案例梳理、資料研究、結構和演示稿製作。

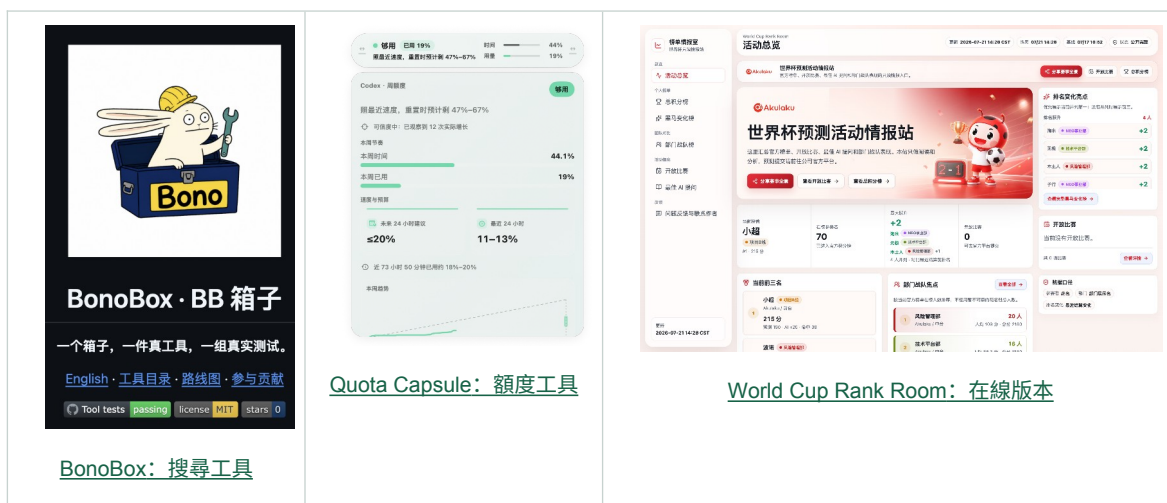
我也會用 Agent 把身邊的小問題迅速做成可以運行的工具或網頁：

- [BonoBox](#)：把網頁、新聞和圖片搜索整理成可安裝的 Python/MCP 工具，包含安裝、測試、錯誤邊界和公開發布；
- [Quota Capsule](#)：讀取本機 Codex 配額信息，結合近期使用速度判斷本周餘量，並在 macOS 選單列提示；
- [World Cup Rank Room](#)：把內部世界盃活動的排名、隊伍比較和 AI 輔助問答做成可以直接訪問的網站，並使用獨立數據和公開部署。

這些項目大小不同，工作方式卻很接近：先把問題說清楚，確定使用邊界和驗收方式，再讓 Agent 加速實現、調試、測試和發布。每個項目都同時保留程式碼、安裝方式、測試結果、失敗處理、版本記錄和給下一位使用者的說明。



萬科財務 AI 實踐分享材料（已打碼）



八、流程從哪裏來

我之所以持續做這些工具和基礎設施，是因為幾段工作經歷一直把我帶回同一個問題：如果流程決定了我每天在做甚麼，那麼又是甚麼在塑造流程？

在安永參與審計和內控工作時，我習慣從結果向前追查：數據和證據來自哪裏，控制是否真正運行，發現異常後由誰判斷和整改。現在負責 ECL 和操作風險、信息系統風險管理，一邊要處理模型、數據與系統交付，一邊也要持續梳理流程中的職責、交接和控制。對流程的關注不是後來使用 Agent 才出現的，它原本就是我工作的一部分。

我逐漸發現，流程很少是一次完整設計出來的。制度要求、系統限制、部門分工、過去發生的問題和個人經驗，都會在裏面留下痕跡。有些安排仍然必要，有些只是長期沒有重新檢查過。Agent 如果直接沿着這些流程加速執行，也會把其中原有的問題一併放大。

Google DeepMind 聯合創辦人兼 CEO 德米斯·哈薩比斯（Demis Hassabis）曾這樣概括 DeepMind 創立時的使命：「Solve intelligence, and then use it to solve everything else.」——先解決智能，再用智能解決其他問題。[1]

哈薩比斯討論的是智能能力本身：先把智能做出來，再把它用於科學和現實問題。我在工作中真正面對的，是這句話的後半段——一個組織如何才能「用智能解決問題」。Agent 已經能夠執行和推理，但組織裏的流程、業務知識、權限和判斷節點往往沒有被說清楚。流程在這裏就是智能進入組織的接口，它決定 Agent 能看到甚麼、可以做甚麼、甚麼時候停下來，以及誰對結果負責。我的工作是先把這些內容整理成 Agent 可以進入的管道，再讓它沿着流程執行和重現。實際運行留下的異常、失敗和反饋，又會回到任務、方法、工具或知識層，繼續修改這條管道。

在實際建設中，我通常會先選取邊界清楚、正在發生的業務問題，驗證系統設計中的關鍵假設；同時保留整體架構、接口和後續擴展路徑，再把經過項目和用戶驗證的能力沉澱為可重用的工具與方法。數據權限、職責分離、運行留痕、停止與回退、人工確認也會從架構設計階段開始考慮。具體採用集中平台、分層能力還是其他建設方式，還需要結合組織的業務目標、現有系統、風險偏好和資源安排共同判斷。

[1] Demis Hassabis, "Demis Hassabis on AI, game theory, multimodality, and the nature of creativity," *Possible*, 9 April 2025: <https://www.possible.fm/podcasts/demis/>

From Copilot to the Agentic Loop

Delivering AI Tools

Building the Supporting Infrastructure

Professional background

My work sits at the intersection of financial risk, systems, data and process. At EY Financial Services Technology Risk, I worked on ITGC, ITAC, and system and data control reviews for banking, insurance and securities engagements. My current role at Akulaku Group covers ECL and operational risk management. The ECL work includes methodology, data processing, system requirements, UAT, finance reconciliation, model updates and support for external review. The operational risk work requires continued review of business processes and their controls.

I first used AI for specific tasks: generating a code fragment or drafting a query. It later became involved in model work, data processing, audit responses and tool development. As the task chain grew, I started paying more attention to how business context was retained, how tools handed work to one another, and whether a problem found in one run would improve the next.

My way of working with AI has changed several times. In 2023, I mainly used browser-based AI tools to generate code fragments for automation and query scripts. I then moved to AI-enabled IDEs such as Cursor, Antigravity and Trae, where AI worked as a copilot during development. From 2025, agents including Claude Code, Cursor CLI, Codex and Hermes began taking on longer tasks across model and tool development, automated workflows, research and documentation.

In 2026, I shifted more of my attention to agentic engineering and to building a loop that could continue across tasks. Drawing on accumulated business and data knowledge, together with my control and process-review experience, I decoupled and restructured existing tools and automations, redesigned their interaction and presentation, and opened selected capabilities in stages according to internal security requirements and real business needs. The projects below trace that move from Copilot to an Agentic Loop inside an existing organisation.

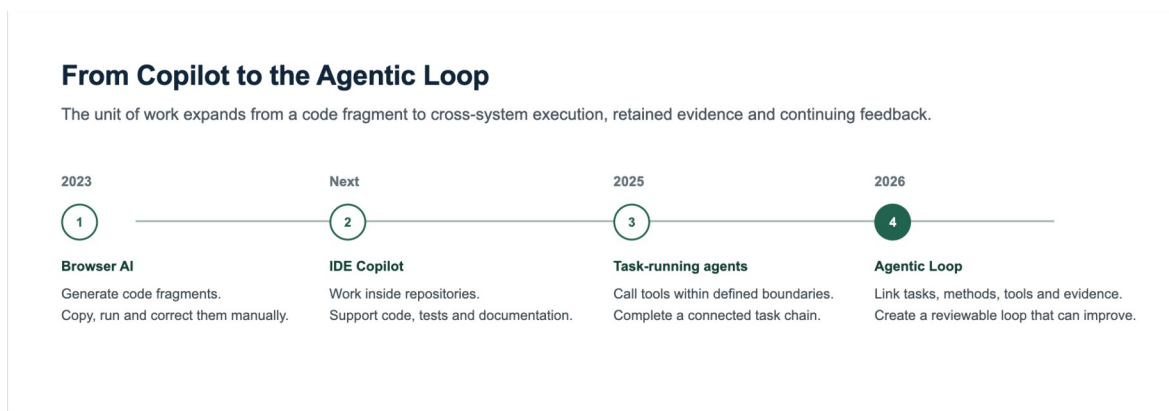


Figure 1. From Copilot to the Agentic Loop

1. From generating code to supporting an ECL update

When I first used ChatGPT, I would give it a narrowly defined request: merge Excel files, process a Word table, collect public information, or write a data-processing script. It produced code; I copied it, ran it, returned the error and tried again. This reduced coding time, but the tool could not see the whole project or check whether upstream and downstream files remained consistent.

The unit of work expanded once Cursor, Claude Code and Codex could operate inside a repository. The 30 June 2026 Group ECL update is my most complete example so far. It covers four lending portfolios in Indonesia and the Philippines: Indonesia BNPL, Indonesia Cash, Philippines BNPL and Indonesia productive lending. The work begins with finance scope and EAD reconciliation, then proceeds through order-level data construction, PD, LGD, EAD, staging, forward-looking adjustment, Group ECL aggregation, result analysis, methodology documentation and responses to prior audit questions.

There is no single platform that runs the project from start to finish. Data comes from Superset/Hive and finance files. FY25 reviewed models provide the methodology baseline. Some data processing must run on a server. Python and notebooks handle calculation and presentation. Excel is used for reading, review and management reporting. Git retains code, rules and versions. Generating SQL is only one step in a chain that moves across all of these environments.

I confirm the reporting-date population, methodology changes, data definitions, material operations and final acceptance. Agents perform much of the continuous execution: locating tables and fields, preparing and running SQL, resolving resource errors, constructing intermediate datasets, preparing model inputs, running models, comparing historical results, building workbooks and assembling review materials. Changes to existing table structures, deletion, and writes whose outcome is uncertain still require my confirmation of the exact target before execution continues.

The project is designed to be reviewable. Each formal query retains its SQL, query identifier, execution time, row count, result file and file hash. Model inputs are frozen. A database write is followed immediately by control checks. A client timeout triggers a target-state check before any rerun. Formal notebooks, Group results, methodology documents and audit responses for all four portfolios must be reproducible from the same set of inputs.

This was the first time I managed agents as continuing executors rather than code assistants. The unit of work grew from one function to a complete sequence: once scope is confirmed, continue through data retrieval, calculation, control checks and delivery.

Related artefacts: project control file, data-construction SQL, portfolio notebooks, Group results, analytical workbooks, methodology documentation, responses to prior audit questions and automated acceptance checks.

2. Audit-response architecture: a repeatable issue-resolution pipeline

This architecture pre-dates the 30 June project. During the FY25 ECL audit response from March to May 2026, I found that one audit question could affect methodology, code, data, explanatory documents and the final package at the same time. If an agent started editing as soon as it received a question, investigation, judgement, implementation and review would blur together. Fixing one point could create a new inconsistency elsewhere.

I redesigned the audit-response architecture around four linked stages that cannot substitute for one another:

Each stage has its own inputs, permitted actions, required evidence and exit conditions. The investigation stage cannot quietly edit code, and “fixed” is not a substitute for a judgement on the issue. Method choices and material consequences remain human decisions. These boundaries are embedded in repository rules, scripts, tests and status files rather than left inside a single prompt.

The architecture also reflects what I call the pipeline theory. The agent is an execution node; task context, data, methods, permissions, verification and feedback form the surrounding pipeline. Problems found in operation strengthen that pipeline. A missing pre-change baseline becomes a mandatory comparison. State contamination between models leads to isolated execution. A notebook that appears to calculate but does not use the calculated value leads to dependency-chain checks and perturbation testing.

Question identifiers, processing windows and queuing rules were added later to preserve state, support parallel work and handle interruptions. They were operational improvements to the architecture, not its starting point. The 30 June project reused the same approach to task state, baselines, reruns, reconciliation and acceptance.

Related artefacts: overall audit-response architecture, supporting SOPs, execution rules and control scripts, baseline and impact comparisons, pseudo-computation tests, independent final review and a visual workflow.

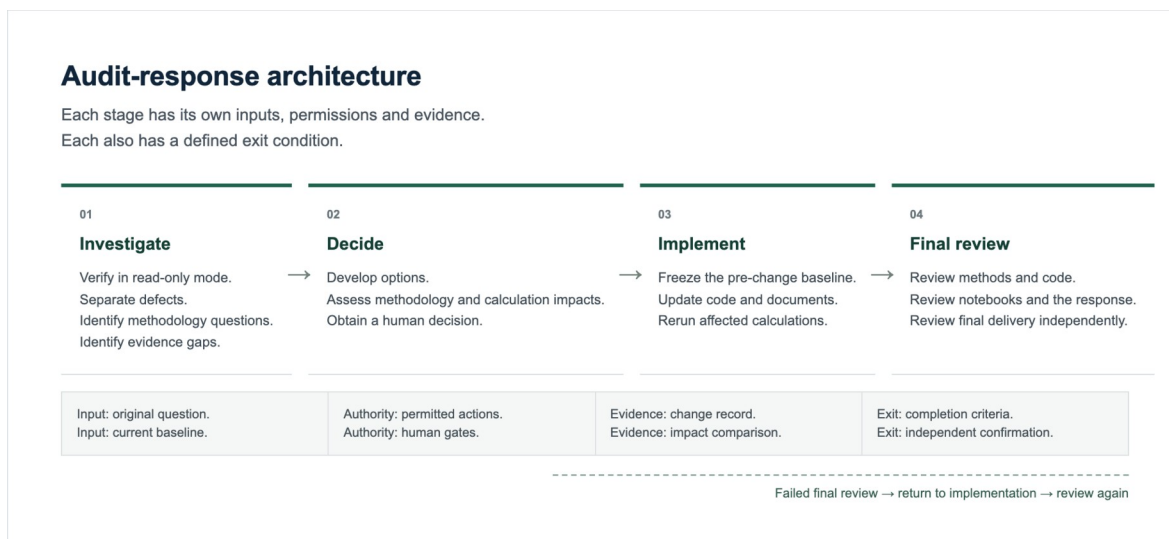


Figure 2. Audit-response architecture

3. Extracting a reusable Superset query tool from the ECL work

ECL updates and asset-quality analysis repeatedly require table discovery, SQL preparation, query execution, error handling and result retention. Agents could already help interpret a question and prepare SQL, but a person still had to copy it into Superset. Execution status and query results did not naturally flow back into the next analytical step, so the same capability remained scattered across projects.

I extracted query execution from the ECL workflow and developed `query-superset`. The agent interprets the business question and prepares SQL. The tool uses the employee's existing Superset identity and permissions to execute read-only queries, then returns errors, results and run records to the agent. The design continues to use the company's approved data-access channel; it does not create a separate direct database connection for automation.

I handled requirements, architecture, security boundaries, scripting, tests and installation. The result was packaged as a reusable Skill for different agent projects, with internal guidance covering sign-in, query execution, common errors, result retention and capability boundaries.

This was the first time I extracted an atomic capability from a complete business project. The agent could now do more than "write SQL": it could complete the query and continue with the returned result. The tool was later made available internally through the company Wiki and an installation package.

Related artefacts: the `query-superset` Skill, Windows query client, tests, installation package, user guide and capability-boundary documentation.

4. From internal use to a higher level of architecture

Colleagues in several departments began using the released tool in their own work. The internal Wiki and supporting material explained installation and operation, as well as the appropriate scope, the controls already provided by the company, and the points at which a user should stop.

As use widened, questions quickly moved beyond "how do I retrieve the data?" to data cleaning, analysis, business knowledge and delivery. The query tool solved access to data. It could not decide an analytical definition, select an investigation method or explain a result. Adding more query features would not resolve those problems.

I therefore revisited the relationship between data retrieval, data analysis and business judgement. Execution tools, analytical methods and factual sources were separated. That step moved the work to a more abstract architectural level and led to the direction I initially called the "large infrastructure" project.

WINDOWS

SUPerset

CURSOR SKILL

READ-ONLY

复用现有权限
 继续走 Superset、公司网络和员工自己的账号，不另开数据库直连。

保持只读边界
 只执行一条只读 SQL，保留结果、错误和查询编号。

把安装交给 Cursor
 员工只需下载、解压并复制部署指令，Cursor 完成配置和实测。

Evidence excerpt. Existing permissions and a read-only boundary

GOVERNANCE

TRAC

ACCESS CONTROL

TEST EVIDENCE

① 这份长文记录方案背后的治理判断：为什么选择复用现有通道、哪些控制可以复用、哪些不能想当然，以及测试证据目前能证明什么。

Evidence excerpt. Control and governance reasoning behind the design

为什么 Wiki 不应该只靠手工复制粘贴

手工维护 Wiki 的问题：

- 格式不一致：每次粘贴排版都不同
- 版本失控：不知道哪个版本是最新的
- 无法审计：谁改了什么，为什么改
- 无法批量更新：几十个页面的统一修改要手动逐页操作
- 敏感信息泄露风险：手工复制容易带入内部数据

Evidence excerpt. Engineering the internal Wiki release process

“I tried one query. It worked.”

Feedback after a colleague completed a live query. The discussion then moved from retrieval to analysis, MCP and business-context input.

Live user feedback and the analytical questions that followed

5. The Bono Agent capability and knowledge foundation

In July 2026, I reorganised the earlier work as an internal project formally called the Bono Agent Capability and Knowledge Foundation. It no longer extended one tool. It separated the different elements an agent needs to complete a business task.

The resulting architecture has four layers:

Task layer: business work remains within its project

The task layer currently carries Group ECL, PD assessment and monthly forecasting work. Each project retains its own entry point, AGENTS.md, project control file and handover state. Changing task information is not copied into the general plugin. insight-router provides a common entry point: it first fixes the original question, subject, period, population, reader, output and risks, then selects the shortest complete route through the required capabilities.

Each use receives a `usage_id` that ties together the Skills actually called, the result and subsequent feedback. At closeout, the Router treats project state, method or tool defects, and knowledge candidates separately. Improving a Skill or logging feedback cannot be mistaken for completing the original business task. One of the next tests is whether a new user can complete a task from the project entry point and existing instructions without my continuous explanation.

Method layer: recurring working methods become Skills

The method layer now contains a connected set of capabilities. `prepare-analysis-data` defines population, period, grain, keys, mapping, control totals and data snapshots. `investigate-data-change` handles anomaly and trend investigation, positive and negative branches, competing explanations and stopping conditions. `deliver-analysis` turns

verified results into an explanation a business reader can use. `deliver-workbook` defines the reading order, numeric meaning, evidence entry points and acceptance checks for Excel deliverables. `insight-router` handles cross-layer routing and task closeout.

These Skills are more than prompts. Each includes reference methods, task templates and completion criteria. Change investigation also has a separate depth validator that checks whether the original question was preserved, both increases and decreases were covered, identified causes reconcile to the total change, and stopping is supported by evidence. Current work prioritises validation in ECL, PD assessment, asset-quality investigation and Excel delivery. Further Skills are added only when recurring business needs and the architecture support them.

Atomic capability layer: reuse approved tools

The atomic layer performs bounded actions. Existing capabilities include Superset queries, company Wiki access, local files and browser operation, together with the native document and Excel capabilities of Codex. Bono Insight manages when to call them, what evidence to retain, and how to judge completion. Execution is delegated to the database, browser and Excel capabilities already available in the host environment.

For example, `query-superset` continues to use the company's identity, permissions and query channel. `deliver-workbook` defines workbook structure and acceptance, while native Excel tooling creates, edits and recalculates the file. A real-time service is packaged as MCP only when cross-project reuse, permission design, maintenance cost and recovery requirements justify it.

Evidence layer: maintain facts separately from methods

The evidence layer contains business-knowledge entry points, data dictionaries, source files, query results, run records, reader preferences and project evidence. Company business facts are not packaged into the distributable plugin. They remain in approved host projects and are read when needed through capabilities such as `business-knowledge`. An important item records not only what it is and where it is stored, but also the applicable entity, grain, effective period, source, current status and level of confirmation. This prevents an agent from treating an old definition as a current fact.

At the current scale, Git-managed structured text and source evidence provide workable version, provenance and effective-period control. A new fact first enters as a knowledge candidate. It becomes applied knowledge only after the authoritative entry is updated and verified. If team use and real-time demand grow, a vector database or long-running knowledge service can be assessed. Near-term work focuses on a clearer business-context interface and different retention rules for formal evidence, rebuildable cache and temporary output.

How the layers connect

Calls normally move down from the task while evidence and state return upward. The Router fixes the task contract and selects methods. Methods call databases, Wiki, files or Excel capabilities approved by the host project. Results and sources enter the evidence layer. The workflow then returns to the original question for validation and delivery. The Workbench is the operating platform, `AGENTS.md` holds cross-layer safety and governance rules, and the Plugin is the installation and distribution form. They are not additional business layers.

A local feedback recorder sends capability defects and knowledge candidates into separate state flows. A public GitHub Issue can be created only from redacted content that has been manually reviewed. Repository tests check plugin structure, feedback state, public-content boundaries and investigation depth. This allows a correction made in one task to improve later work without allowing the agent to rewrite rules and publish them automatically.

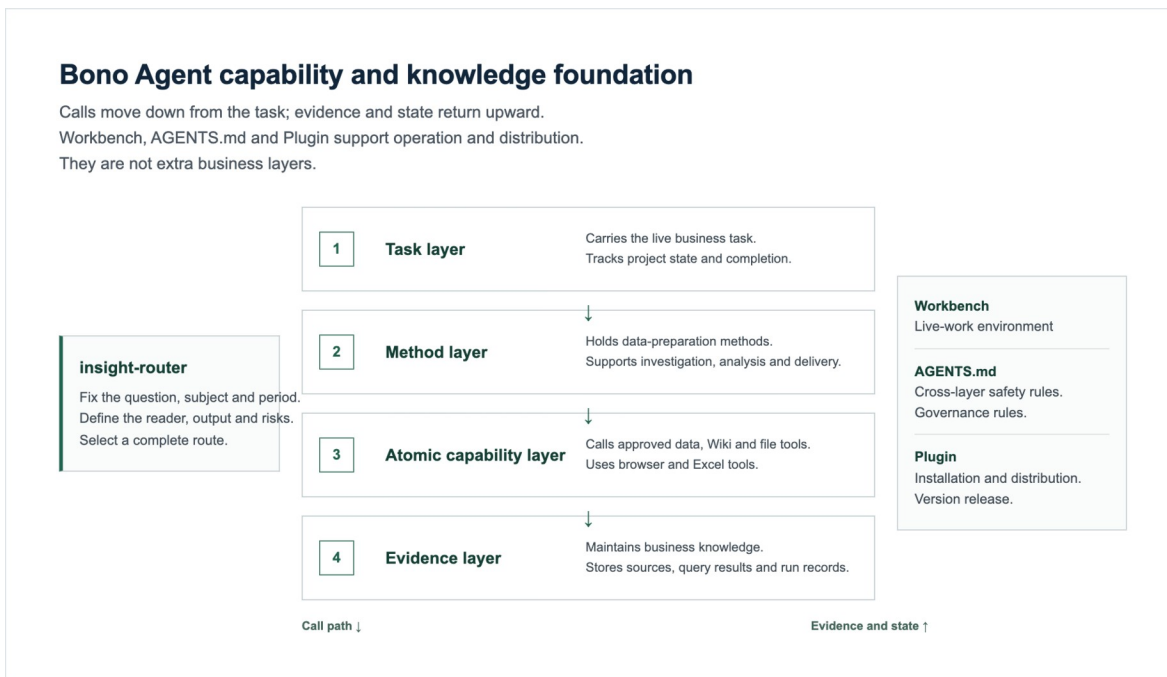


Figure 3. Four-layer capability and knowledge foundation

Returning to 0630: architecture in a live delivery

The architecture was not built outside the business and then introduced afterwards. The 0630 ECL update, supporting the Group's IPO-related financial preparation, is its most complete live run to date. Superset querying, business knowledge, data-preparation methods, the audit-response pipeline and acceptance rules let agents move through data construction, model execution, cross-checks, anomaly investigation, result workbooks and methodology materials for four portfolios from one task entry point. Confirmed field meanings, methodology baselines, file locations and completion criteria do not have to be explained again in every conversation. Queries, model inputs, run results and review evidence remain on the same evidence chain. These capabilities are supporting the current full refresh of model data and results.

The 0630 project also exposed what remained rough in the architecture and directly shaped the task and method layers. Updating four portfolios together required tighter control of reporting date, population, portfolio scope, methodology version, material operations and acceptance state. Population reconciliation, date semantics, keys, mapping and snapshots became part of `prepare-analysis-data`. The need to explain both positive and negative movements rather than net change alone led `investigate-data-change` to add quantified attribution, competing explanations and stopping conditions. The requirement that notebooks, Group results and workbooks be reviewable by someone not involved in development further shaped `deliver-analysis` and `deliver-workbook`.

The same project continues to improve the evidence layer. Business definitions, sources, field limitations, reviewed methodologies, finance control totals, formal queries and run records for all four portfolios return to their authoritative locations with applicable periods and confirmation status. The result is a two-way loop: existing infrastructure reduces repeated searching and rework, while problems encountered in the project become new task controls, method changes or verified knowledge for the next run.

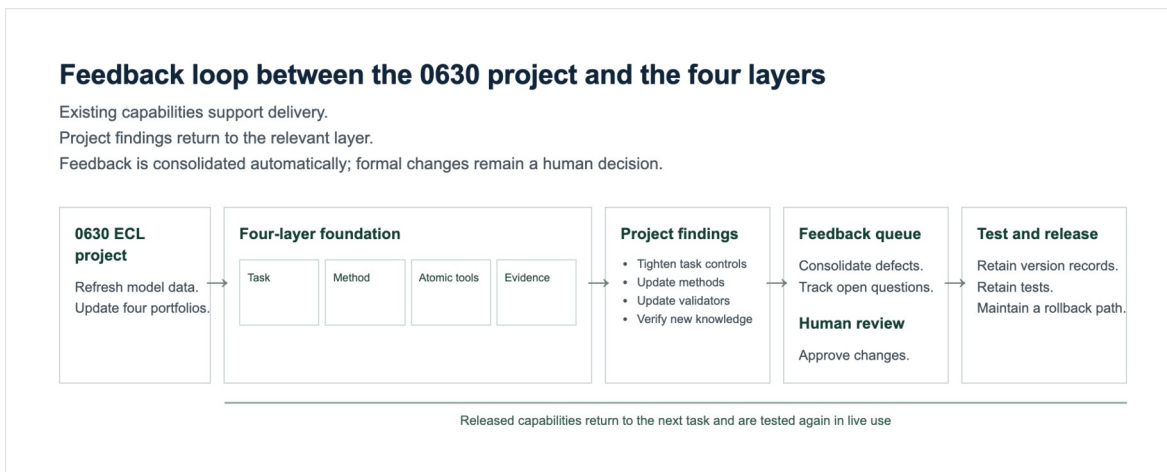


Figure 4. Feedback loop between the 0630 project and the architecture

6. Bono Insight: packaging the methods as a plugin

After validation in the 0630 project and other live tasks, I organised the stable methods, templates, validation scripts and safety boundaries as Bono Insight. I did not package the whole Workbench, company knowledge or private data capabilities with it. The current release is an OpenAI/Codex Preview plugin. Support for other agent environments is planned as thin adapters around the same core methods rather than separate copies of the Skill content.

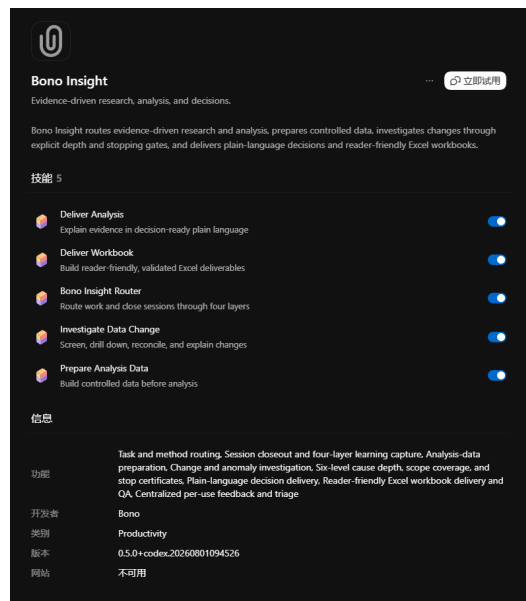
Bono Insight contains installable Skills, reference methods, validation scripts, safety guidance and a feedback mechanism. The release package is built from internal authoritative sources, and a Source Map records the relationship between internal methods and distributable content so that the two versions do not drift independently. The plugin is tested locally and through GitHub Actions, and can now be installed and called in Codex.

Plugin packaging gives the four-layer architecture a continuing feedback path. The Router opens a separate record for each use. At closeout, the Skills used, completion state, capability defects and new knowledge candidates enter a local feedback queue. Scheduled jobs consolidate these records and surface recurring defects, open tasks, knowledge awaiting verification and architectural questions that need review.

The queue does not edit the plugin by itself. A scope or acceptance problem returns to the task layer. Repeated data-preparation, investigation or delivery problems return to the relevant method Skill, reference or validator. Failures in query, Wiki, file or Excel operations return to the corresponding atomic capability. A new business fact remains a knowledge candidate until its source, applicable period and scope are verified. Formal changes still pass tests, version records and release. “Self-improvement” therefore has a defined boundary: the system captures and consolidates feedback; a person decides what should enter the formal version.

The release remains Preview. Before a stable release, the priority is new-machine installation, cold start by a non-author user, continuity across tasks, whether the Router chooses the shortest complete route, and upgrade, rollback, retention and clean-up. The scheduled review and human decision mechanism also needs continued evidence that it reduces recurring errors instead of accumulating maintenance work. Potential later directions include thin adapters for other agent platforms, more redacted examples and evaluations, and real-time services or MCP only when repeated demand supports them.

At this point, “AI infrastructure” has a concrete meaning in my work: separate the task, the method, the execution capability and the evidence so that an agent knows where to obtain facts, how to work, when to stop and who confirms the result.



Bono Insight installed and available inside Codex

7. Knowledge sharing and other rapid delivery cases

I also document working methods for other people. At the invitation of a friend working in finance at Vanke, I helped prepare internal material on the practical impact of AI on finance work. It follows a real asset-quality reporting workflow and shows how data retrieval, analysis, reporting and review connect. It also covers pseudo-computation, hallucinated citations, information security and human judgement. My friend delivered the session; I handled case design, research, structure and presentation production.

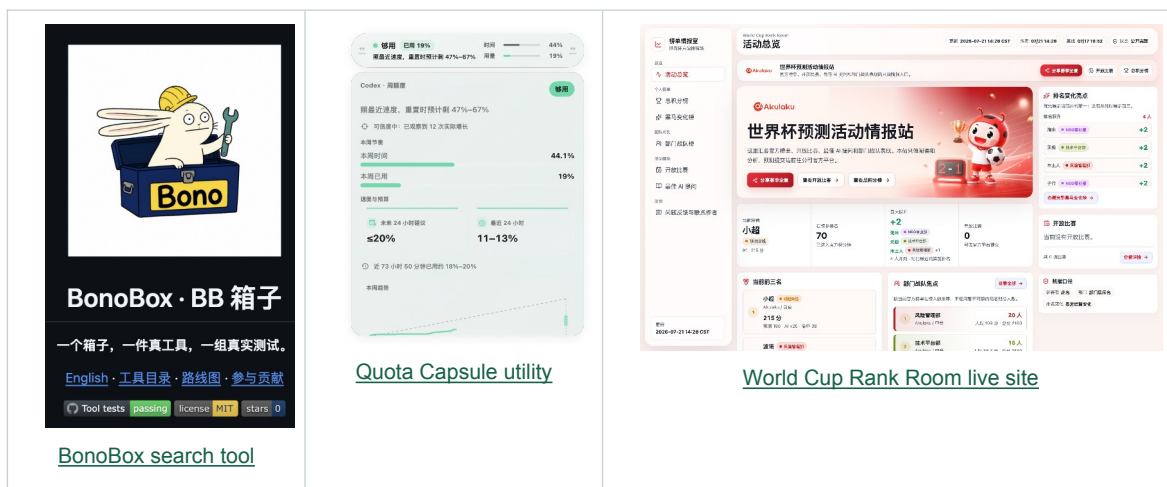
I use agents to turn smaller problems into working tools and websites:

- [BonoBox](#) packages web, news and image search as an installable Python/MCP tool, with installation, testing, failure boundaries and a public release.
- [Quota Capsule](#) reads local Codex quota information, combines it with recent usage pace, and shows the expected weekly runway in the macOS menu bar.
- [World Cup Rank Room](#) turns an internal World Cup activity into an accessible site for rankings, team comparison and AI-assisted questions, using a separate public dataset and deployment.

The projects differ in size, but the working pattern is similar: define the problem, usage boundary and acceptance conditions, then use agents to accelerate implementation, debugging, testing and release. Each project retains code, installation instructions, tests, failure handling, version history and guidance for the next user.



Vanke finance AI-practice sharing material (redacted)



8. Where processes come from

I keep returning to one question through these projects and through my earlier work: if a process determines what I do every day, what shaped the process itself?

At EY, audit and control work trained me to trace a result backwards: where the data and evidence came from, whether a control actually operated, and who judged and remediated an exception. In my current ECL, operational risk and information-system risk work, I deal with model, data and system delivery while continuing to review responsibilities, hand-offs and controls. My interest in process did not begin with agents. It was already part of the job.

Processes are rarely designed once as a complete whole. Policy requirements, system constraints, organisational boundaries, prior incidents and individual experience all leave marks. Some arrangements remain necessary; others have simply not been reconsidered for a long time. An agent that accelerates the process without examining it will also amplify its existing weaknesses.

Google DeepMind co-founder and CEO Demis Hassabis described DeepMind's founding mission as: "Solve intelligence, and then use it to solve everything else." [1]

Hassabis was talking about building intelligence itself and then applying it to scientific and practical problems. My work deals with the second half of that statement: how an organisation can actually use intelligence to solve problems. Agents

can already execute and reason, yet organisational processes, business knowledge, permissions and decision points are often implicit. Process is the interface through which intelligence enters an organisation. It determines what an agent can see and do, when it must stop, and who remains responsible for the outcome. I organise these elements into a pipeline an agent can enter and reproduce. Exceptions, failures and feedback from live runs then return to the task, method, tool or knowledge layer and change the pipeline.

In practice, I select current business problems with clear boundaries to test the important assumptions in a system design. I retain the overall architecture, interfaces and expansion path while turning capabilities proven in projects and real use into reusable tools and methods. Data access, segregation of duties, run records, stopping and rollback, and human confirmation are considered at the architecture stage. Whether the right implementation is a central platform, layered capabilities or another form depends on the organisation's objectives, existing systems, risk appetite and resources.

[1] Demis Hassabis, "Demis Hassabis on AI, game theory, multimodality, and the nature of creativity," *Possible*, 9 April 2025: <https://www.possible.fm/podcasts/demis/>